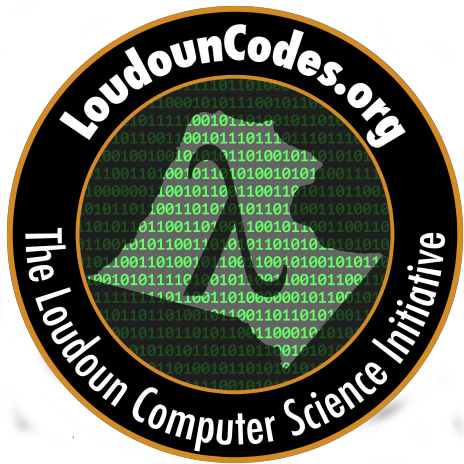




Karel

Creative Project Lessons

LoudounCodes

Java 18 · jGRASP

Classroom review edition · October 2026

© 2026 Bock / LoudounCodes. Lesson text and original illustrations: CC BY 4.0. Java code and arena screenshots: GPLv3.
Logo excluded from CC BY. Attribution does not imply endorsement.

This page intentionally left blank.

Lesson plan

The classroom archive includes an independent starter folder for each lesson. Open its jGRASP project and complete the TODOs. The appendix contains complete reference programs.

Start with the API orientation, then choose the features your own project needs. Print the selected lesson pages; complete program listings and teacher notes follow separately.

0. Terminology: Arenas, Locations, and Directions

1. Robot and beeper colors
2. Interfaces and event scoring
3. Build walls, supplies, and doors
4. Pacing and STEP mode
5. Directions and protected behavior
6. Load maps and change levels
7. Custom items and robot rendering
8. AlphaBot lettering and infinite supplies
9. MapDataSource design extension



This page intentionally left blank.

About this library and curriculum

LoudounCodes JKarel extends the familiar Karel classroom environment into a setting for student-designed games, simulations, and visual projects. It is a modernization of the original Fairfax County Public Schools Karel library, with clearer API names, support for newer Java versions, and additional ways to construct and control the world. This edition targets Java 18 and retains the Ant build and jGRASP workflow used in the classroom.

Karel gives students a foundation in actions, decisions, and visible consequences. Once they master the original assignments, their questions grow: Can a door open? Can teams have different colors? Can a robot's pickup change the score? This library makes those questions part of the next stage of learning Java.

A clearer interface and a richer world

The older API organized the world around a `Display`. That name suggests a screen: something that shows the action. An `Arena` names the place where the action happens. A robot belongs in an arena alongside walls, supplies, and other robots; the display presents that world to the viewer. The distinction helps students reason about the simulation separately from its presentation. Calls such as `Arena.addNorthWall` express an operation in the world they are constructing.

`Pacing` makes a similar distinction. A display's "speed" leaves the reader to interpret what is moving and what the number means. An arena's pace describes how its activity unfolds. Like a pace car governing a race, `pacing` regulates the rhythm of the action. Named values such as `Pacing.SLOW` and `Pacing.STEP` make that intent visible: students control when actions proceed while the rules of movement stay the same.

These choices establish a mental model. Object-oriented programming lets us represent a real or imagined world through objects with recognizable roles, responsibilities, and relationships. A coherent vocabulary helps programmers reason about the system and communicate their intentions. In a small classroom program, that clarity makes the next line easier to write. In a project maintained by a dozen programmers over several years, the same clarity supports shared understanding as people and requirements change. Small API decisions become part of the structure on which larger programs depend.

Students can build walls and supplies programmatically, open doors by removing walls, color robots and beeper trails, and change maps while retaining a player. Listeners let a scorekeeper respond to moves and pickups. Custom rendering gives items and robots their own appearance. Students write the rules and decide what each event means in their game.

Java concepts with a visible purpose

The lessons introduce the concepts needed to use these capabilities. Enums describe directions and pacing. Subclasses give robots additional behavior. Interfaces let one object observe another, and rendering methods connect an object's role in the world with its appearance. These ideas arise from concrete project needs, so students can examine both the Java code and its visible result.

The library's internals also provide material for more advanced Java courses. Teachers can trace an action from the public API into the model, examine how listeners receive notifications, or discuss the responsibilities of an abstract item and its subclasses. The introductory lessons stay close to the public teaching interface; later work can use the implementation to investigate design choices, boundaries, and opportunities for improvement.

Using the curriculum

This sequence is intended for teachers and students who already know Karel or an earlier version of JKarel. The opening lesson explains the changes in terminology and API. The remaining lessons introduce individual capabilities, with a brief concept explanation, exercises, and complete commented programs in the appendix. The jGRASP setup instructions and teacher notes support classroom use.

Follow the sequence or choose lessons for a particular project. The final map-interface lesson is an advanced design exercise connecting a declared contract to a student's implementation. The aim throughout is to move from solving a supplied Karel problem to designing a program students can explain, test, and extend.

Set up in jGRASP

Use JDK 18 or later. The library targets Java 18; jGRASP's own minimum Java version is a separate requirement. The teacher builds once with ant and gives students out/jkarel-1.0.0.jar. Students do not need Ant or JUnit to run these labs.

In jGRASP, open **Settings** → **PATH/CLASSPATH** → **Workspace**, select **CLASSPATH**, then **New** and browse to that JAR. Apply the setting. If your classroom uses jGRASP projects, choose the project scope instead of Workspace. Remove an older JKarel JAR entry so the compiler and runner see the same library. Keep the original FCPS library available for assignments that still import edu.fcps.karel2; these labs import org.loudouncodes.jkarel.

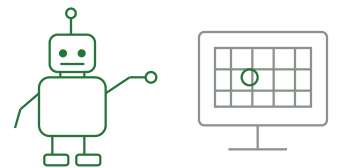
Copy the example .java file into your own working folder, open it in jGRASP, compile, and run. The file name must match its public class name. Extra dependency JARs are not needed. The level-changing lesson needs its two map files copied beside the Java example; the other lessons build their own scenes.

- package org.loudouncodes.jkarel does not exist: check the CLASSPATH entry.
- UnsupportedOperationException: check the Java used to run in jGRASP; it needs 18+.
- A wall exception: read the failing line and use the appropriate ...IsClear() sensor before that move. The robot remains where it was.
- STEP pacing waits for Enter in the run console. FAST is used in the examples.

Teacher command-line equivalent, from the repository root:

```
ant
javac --release 18 -cp out/jkarel-1.0.0.jar -d build/labs examples/java/TeamTrails.java \
    examples/java/EventScoreboard.java
java -cp out/jkarel-1.0.0.jar:build/labs TeamTrails
java -cp out/jkarel-1.0.0.jar:build/labs EventScoreboard
```

On Windows, replace the runtime classpath's : separator with ;. Close each arena window before running the next example.



Lesson 0: Meet LoudounCodes JKarel

New terminology: Arenas, Locations, and Directions

You already know Karel. Learn these names to read the new API and build your own world and game rules.

Arena, Location, Direction

Arena is the shared world; Location is a grid position; Direction is a facing value. Arena methods operate on the scene. Robot methods act on one robot.

Arena: the place where actions happen

Robots, walls, beepers, and game state
Construct the world; perform an action

render

Display: the view of that world

Draw positions, colors, and shapes
Present the result to the viewer

Pacing controls the rhythm of arena actions; it does not change movement rules.

Names guide the model: a robot inhabits an arena, and a display shows that arena.

From a display to an arena

An arena names the place where robots, walls, and beepers interact. `Arena.addNorthWall(4, 3)` reads as an action in that place; `Arena.setPace(Pacing.SLOW)` names how its activity unfolds. These names express the model and make the API more fluent to read. These methods do not support chained calls.

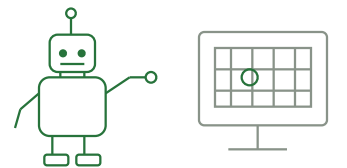
Compare a familiar opening with the new version:

```
// Familiar FCPS-style opening:
// Display.openWorld("my-map.map");
// Display.setSpeed(5);
// Robot karel = new Robot(2, 2, Display.EAST, 2);

// JKarel opening:
Arena.openDefaultMap();
Arena.setPace(Pacing.FAST);
Robot karel = new Robot(2, 2, Direction.EAST, 2);
```

The changes to carry into your own code:

- Import `org.loudouncodes.jkarel` for these projects. Existing assignments that import `edu.fcps.karel2` still use their original library.
- Use `Arena.loadMap(name)` or `Arena.openDefaultMap()` to prepare a scene; `Arena.setSize(width, height)` sets the displayed grid.
- `Direction` is a `Direction`, not an integer. An arbitrary number cannot be passed as one. Choose a named Pacing value; there is no numeric speed conversion rule.
- `getLocation()`, `getX()`, and `getY()` describe a robot's grid position; `getDirection()` describes its facing.



- Infinite beepers use `BeeperStack.INFINITY`. Robots, walls, and stacks are kinds of `Item`; each describes how it is drawn.
- The normal robot verbs and sensors still work. Teach a subclass `turnRight` by calling `turnLeft` three times.
- `Arena` is the front door; `ArenaModel` holds the scene's state. `Arena.getModel()` exposes custom items and removable walls.

Predict and run

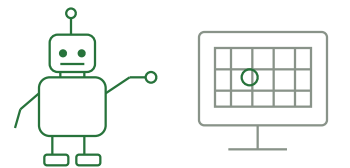
Compile `WelcomeArena.java` with the [jGRASP setup](#). Trace its two moves on paper before you run it. Expected output is location `[3, 3]`, direction `NORTH`, and inventory `1`. The dropped beeper stays at `[2, 2]`.

Change the initial direction to `NORTH`. Predict the new endpoint before running. Then change the pace to `Pacing.SLOW`. Explain why timing changes but the route and inventory do not. Read the console if an action fails; this version reports an exception at the failing line instead of quietly ending the program.

Show what you learned

Write a five-line opening for your own project. Explain which calls affect the arena and which affect one robot. Translate an opening from your own earlier work into the new names, and explain why a direction and a location are different kinds of information. Leave unrelated starter classes in their original package.

Teacher prompt: read an `Arena` call aloud as a sentence, then distinguish the front door from the stored scene. Leave `Swing` internals for a later course.



Lesson 1: Team trails

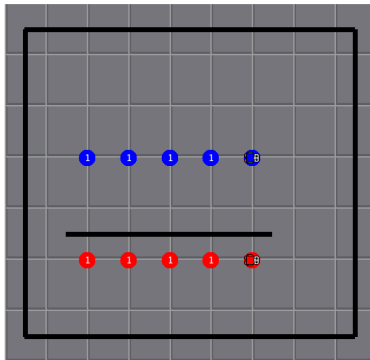
New capability: Robot/Beeper Colors

How could a player tell which team owns a trail? Build a small arena with two colored robots, then use the colors and walls in a project of your choice.

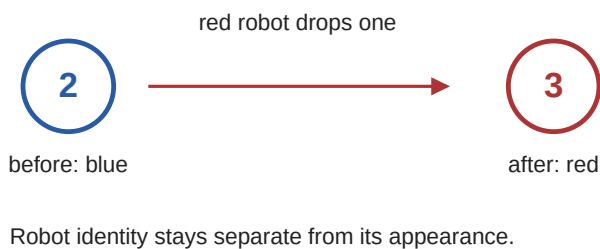
You should already be comfortable constructing a Robot, calling its methods, using loops, and checking a sensor. After this lab you should be able to create walls in code and explain how a robot's color affects its dropped beepers.

Robot/Beeper Colors

A robot has a drawing color. When it drops a beeper, that color applies to the whole stack. Color shows appearance; your program must keep team identity separately.



One color belongs to the whole stack



Completed TeamTrails at left. At right, a red drop recolors an existing blue stack.

Predict, run, inspect

Open `TeamTrails.java` using the [jGRASP setup](#). Before running, sketch the grid. Predict the robots' final locations, their remaining inventories, and the colors of the beepers. Run it and compare with your sketch.

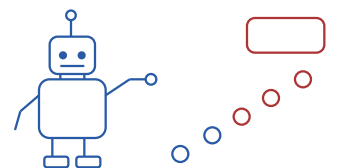
Expected result: a red trail on row 2 and a blue trail on row 4, each covering columns 2 through 6. Both robots end in column 6, with no beepers left. A wall separates the lanes. The program builds its own outer boundary.

Color comes from `java.awt`. Use `Color.RED`, `Color.BLUE`, or new `Color(80, 160, 220)`; each red/green/blue component is an integer from 0 to 255.

```
robot.setColor(Color.BLUE);  
robot.putBeeper();
```

The drop colors the **whole stack** at that location. If a red robot later drops onto a blue stack, the stack becomes red. Picking up a beeper preserves the color of the remaining stack. Picking it up does not change the robot's color.

`Arena.addNorthWall(x, y)` places a unit wall immediately north of that cell. The East, South, and West methods work the same way. Adjacent calls can build a longer wall. `Arena.setSize` changes the displayed grid; it does not enclose it with a full boundary. Build boundaries explicitly, as this example does.



Lesson 1: Exercises

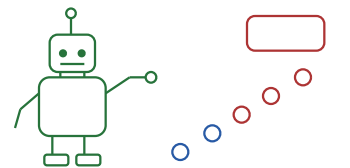
1. Change both team colors. Explain why you need an import for `Color`.
2. Change a robot's color halfway through its trail. Predict which beepers change.
3. Add a second divider with a gap. Write a robot route through the gap using turns and sensors; keep the route inside the boundary.
4. Have the robots meet and drop on the same cell. Test the whole-stack color rule.

Choose one extension for your project: colored territory, breadcrumbs through a maze, team bases, or a drawing made by several robots. Color is appearance, not a team identifier: keep a separate variable or robot reference for game rules.

Show what you learned

Show your arena and the code that builds it. Explain a north-wall coordinate, why a move is safe, and the result of two different colored drops on one cell. Give a prediction and observation for one change you made. Your program should finish its planned route without a wall exception or an empty-inventory drop.

Teacher prompts: ask why the loop avoids moving after the final drop; have a student point to the wall that a left-facing sensor would detect. Extra challenge: extract a reusable `buildRoom` method that takes a location and dimensions.



Lesson 2: Let the arena tell the scoreboard

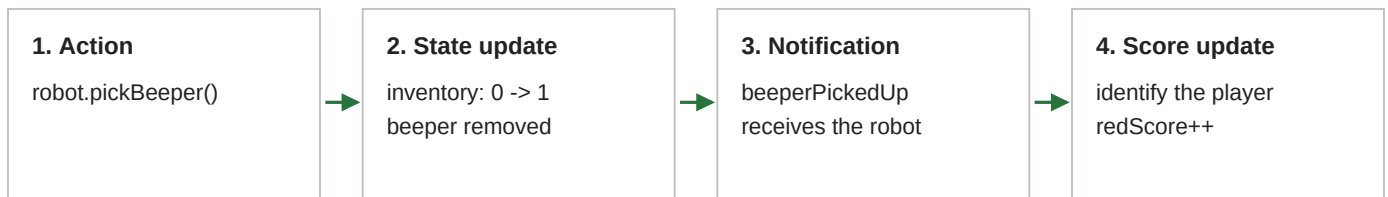
New concept: Listeners

A game needs to count pickups and moves. Should every robot have to know about the scoreboard? Use an interface to let a separate observer react to arena events.

Complete Team trails first. You should know fields, methods, conditionals, and object references. This lab introduces an interface with default methods and the observer pattern: one object performs an action; another receives a notification.

Listeners

A listener is an object registered to receive notifications. The arena calls its methods when actions happen. The listener observes those actions without taking over the game loop.



Main loop owns the turn. The listener observes a completed pickup.

Callback order matters: the listener sees the updated inventory, then updates its own score.

Predict and run

Open `EventScoreboard.java` using the [jGRASP setup](#). Read `takeTurn` and `Scoreboard`. Predict the number of pickup callbacks and move callbacks before running.

Expected console results progress through red/blue scores of 1/0, 1/1, 2/1, and 2/2. The final line is `Final: Red 2, Blue 2, moves 8`. Both robots finish at column 6; each holds two beepers. The main loop drives the turns. The scoreboard never moves a robot.

```
Arena.addListener(scores);
```

This registers an object implementing `ArenaListener`. Its default methods do nothing, so the class overrides only the events it wants. `@Override` lets the compiler check the method name and parameter type. For a pickup the library calls `beeperPickedUp(Robot robot)` after updating the stack and inventory. Comparing `robot == red` tests whether this is the exact red robot object; its appearance could change while its identity stays the same.

Experiments

1. Move one beeper farther down its lane. Predict which turn produces its pickup.
2. Count drops separately by overriding `beeperDropped`. Add a drop in `takeTurn` after a pickup. Watch the scores: decide whether a pickup of a previously dropped beeper should award another point in your game.
3. Add a third robot and track it explicitly. Color alone must not determine score.
4. Remove the listener before the final turn. Explain why the robot still moves but that action no longer changes the observed counters.



Choose a project use: count maze steps, track food pickups, record a move log, or announce when a player has collected enough items. Write the win check in the main game loop after taking a turn; the callback can update a flag or counter.

Event scoreboard: callback rules

- Callbacks run immediately on the thread that performs the action. Keep them short. Do not start another movement or a game loop from a callback.
- `robotMoved` reports completed forward/backward movement; turns have no event.
- A robot drop produces `beeperAdded` followed by `beeperDropped`. Use the latter to count player drops; counting both would count one action twice.
- `robotAdded` runs before the constructor returns. Register scoring after player creation, as this example does; do not depend on subclass fields in that event.
- `wallCollision` observes an attempted blocked move before an exception is thrown. Sensors are still the right way to plan safe movement.
- `mapLoaded` fires after loading. Map loading preserves existing robots and listeners while replacing walls, beepers, and custom items.
- There are no keyboard-input, timer, turn, or color-change callbacks yet.

The `finally` block removes this listener even if a robot action fails. It does not reset the scoreboard, arena, or robots; those are separate responsibilities.

Show what you learned

Demonstrate one new counter or announcement. Explain the difference between calling a robot method and receiving a callback, why pickup sees the new inventory, and why the game loop belongs outside the listener. Include the predicted and observed callback counts for your change.

Teacher prompts: ask the student to trace one `move()` and one `pickBeeper()` into their callbacks. Extra challenge: add a second listener that records a text log; explain how it can coexist with scoring without modifying `Robot`.



Lesson 3: Build a room with a door

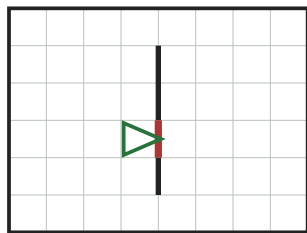
New capability: Programmatic Walls and Beepers

Maps need not be fixed before a game starts. Construct walls and beepers with loops, then let the program open a door. Use coordinates and conditions you already know to describe the world you want.

Programmatic construction

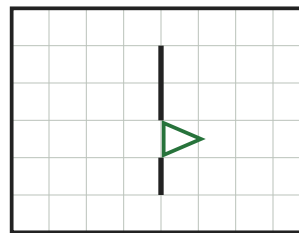
The program can place walls and beeper stacks while it runs. Loops describe repeated layout. Keep a wall object reference when the game needs to remove that specific wall.

Door closed



frontIsClear()
false
player [4, 3]

Door open



frontIsClear()
true before crossing
player [5, 3]

Colored segment: the stored Wall object removed to open the door.

The boundary and divider remain. Removing one wall changes what the front sensor can see.

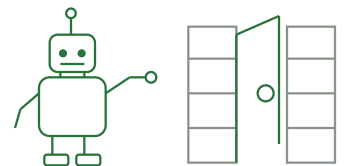
Predict and run

Compile `RoomBuilder.java`. Sketch its 8-by-6 room, divider, and door. Predict what `frontIsClear()` reports before and after the door is removed. Run the example: the first result is `false`; the player then moves from [4, 3] to [5, 3], picks up one beeper, and holds 1. Two beepers remain on that cell.

```
Arena.addEastWall(4, 2);  
Arena.addBeepers(5, 3, 3);  
Wall door = new Wall(4, 3, Arena.VERTICAL);  
Arena.getModel().addWall(door);  
Arena.getModel().removeWall(door);
```

North/South/East/West wall calls describe a wall relative to a cell. Each adds one segment. A loop builds a longer barrier. An east wall at [4, 3] separates columns 4 and 5. `Arena.setSize` changes the visible grid; construct enclosing walls explicitly.

Keep the `Wall` reference when you plan to remove it. Constructing another wall with the same coordinates does not give you the same object. Model-level wall changes appear on the next arena repaint; the following robot action supplies one here. Do not implement a game loop inside a wall event callback.



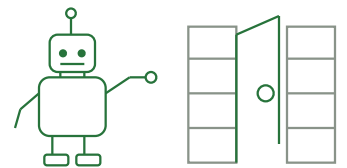
Lesson 3: Exercises

1. Move the gap to another row. Change the player's starting location to match.
2. Keep the door closed until the player holds a key represented by a beeper.
3. Write a `buildRoom` method taking width and height, then build two room sizes.
4. Place food with `Arena.addBeeper` or supplies with `Arena.addBeepers`. Decide when a game should replenish them and express that rule in the main loop.

Show what you learned

Demonstrate a door changing a sensor result. Explain the difference between a wall coordinate and a robot location, and why removing the stored door object works. Your route must check the correct sensor before moving.

Teacher prompt: ask which cell lies immediately west of an east wall. Project extension: represent a maze with rows of characters and translate them into unit wall segments; begin with a small hand-written maze before random generation.



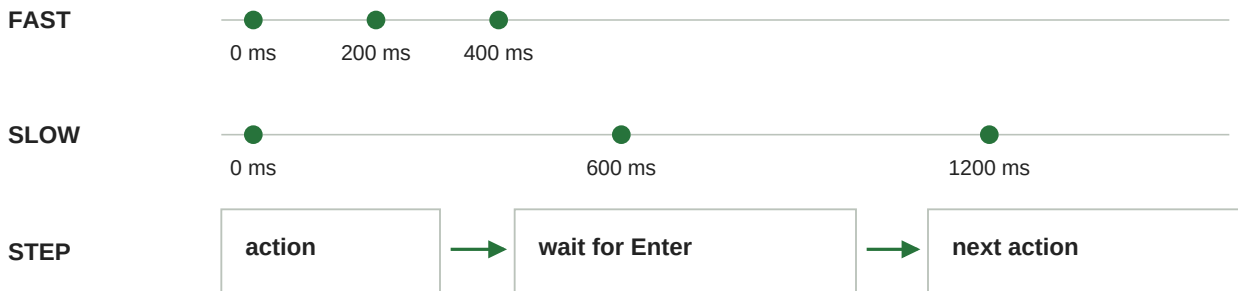
Lesson 4: Predict an action with STEP pacing

New capability: Pacing and STEP Mode

Slow a program down to explain it. Pacing is a named choice that changes how long the program waits between actions, while preserving the rules of the robot.

Pacing

Pacing controls the wait after an action. STEP waits for Enter in the run console. A named enum value selects timing; changing pace does not change the robot rules.



Schematic timing after setup: FAST waits 200 ms, SLOW 600 ms, and STEP waits for Enter.

Predict and run

Compile `PaceProbe.java`. The robot is created at `[3, 3]`, facing east, before the example switches to `Pacing.STEP`.

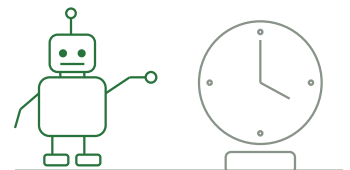
```
Arena.setPace(Pacing.STEP);
robot.move();
robot.turnLeft();
robot.move();
```

For each action, predict location and direction. Focus the jGRASP run console and press Enter when the program waits. STEP pauses after the action changes state and requests a repaint, before that call returns. It does not wait before the state change. The console traces are `[4, 3] EAST`, `[4, 3] NORTH`, and `[4, 4] NORTH`.

Creation, color changes, and several arena operations also use pacing. Setting STEP before constructing a robot can therefore pause in its constructor. For a clear movement trace, finish setup first as this example does. STEP uses console input, so do not combine it with another Scanner reading the same input stream.

Experiments

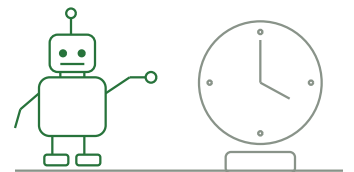
1. Compare SLOW, MEDIUM, FAST, and LUDICRUS using the same three actions. LUDICRUS is the exact spelling in this library. Confirm that the endpoint is identical even when you cannot watch every frame.
2. Insert a beeper drop with an appropriate starting inventory. Predict the inventory at each pause.
3. Pause a short section of your own project, then restore `Pacing.FAST`. Use `Arena.getPace()` to report the current choice.



Show what you learned

Submit a location/direction/inventory trace and demonstrate it in STEP mode. Explain why reducing delay cannot repair a wrong algorithm. For unattended graphical runs, this example accepts `--auto`; actual STEP behavior must be checked interactively in the classroom.

Teacher extension: read `Pacing.java`. Each enum value provides the same `tick` behavior with a different implementation. Name the strategy pattern only after students can explain the observable difference.



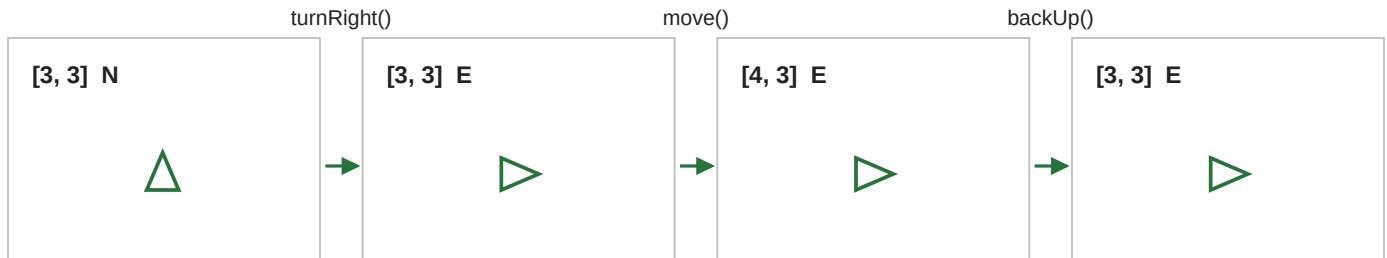
Lesson 5: Directions, sensors, and a retreating scout

New capability: Backing Up; New concept: Protected Methods

A robot can face one direction while checking another. Extend Robot with a retreat method and use typed directions to reason about its options.

Protected movement

Backing up changes position without turning. A protected method is available inside a subclass. Your Scout exposes a public retreat operation that checks the rear sensor before using backUp.



Retreat checks backIsClear() first. Backward movement preserves facing.

Position and facing are different state: a turn changes one, and a retreat changes the other.

Predict and run

Compile [ScoutMoves.java](#). Its Scout subclass starts at [3, 3], facing north. A wall is immediately west of it. Predict its left sensor, planned facing, and final position.

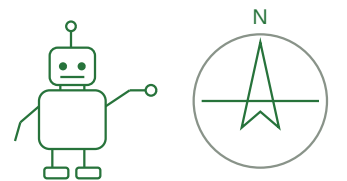
```
public void turnRight() {
    turnLeft(); turnLeft(); turnLeft();
}
public void retreat() {
    if (backIsClear()) backUp();
}
```

Expected results: left clear is false; the planned direction is EAST. After turning right, advancing, and retreating, the scout returns to [3, 3] and still faces east. Backing up changes position without turning.

backUp() is protected: a Robot subclass can call it, but the main program cannot call robot.backUp() on an ordinary Robot. The subclass exposes a meaningful operation, retreat, and checks its precondition. Public side sensors are available on any Robot. Each tests its own side relative to the current facing.

```
Direction planned = scout.getDirection().right();
```

This computes a direction value. It does not turn the scout. left(), right(), and behind() describe relations between enum values. turnLeft() is an action on the robot. Avoid integer arithmetic on directions or assumptions about their numeric order.



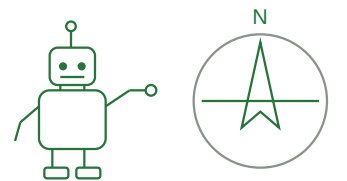
Lesson 5: Exercises

1. Block the cell behind the scout and show that retreat leaves it in place.
2. Give the scout a turnAround method using two left turns.
3. Write chooseExit to inspect sensors and return a Direction. Keep choosing separate from the method that performs the actual turns and movement.
4. Use nextToRobot(other) for a meeting in a cooperative game. It means sharing a cell, not occupying neighboring cells; robots can share a location.

Show what you learned

Explain public versus protected with your own subclass. Demonstrate one method that computes an answer and another that changes the robot. Show safe movement when the front and back have different wall conditions.

Teacher prompt: rotate the paper sketch without changing the world walls and ask which relative sensor now detects the same wall.



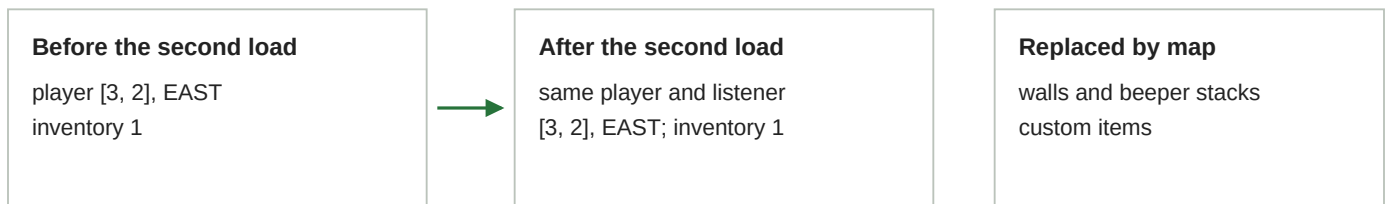
Lesson 6: Change levels while keeping the player

New capability: Map Reloading

Move from one map to another during a program. First decide which state belongs to the map and which belongs to the player.

Map reloading

Loading replaces the map contents while keeping existing robots and listeners. Player position, facing, inventory, and your score variables survive. A level transition must decide which state to reset.



Load changes the scene. The program decides whether to reset player and game state.

MapStages retains the player while level two introduces a wall immediately in front of it.

Set up and run

Copy [MapStages.java](#), `stage-one.map`, and `stage-two.map` from `examples/maps` into the same student working folder. In jGRASP, compile and run the Java file from that folder. These maps were written for this lab; they are small XML files you can read and edit.

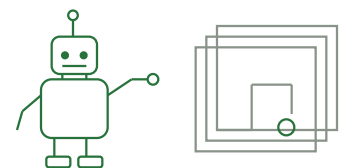
```
Arena.loadMap("stage-one.map");  
// Take a turn using the current map.  
Arena.loadMap("stage-two.map");
```

The example uses the current working folder by default. From the repository root, run it with the argument `examples/maps` to select the supplied map folder. If a map is missing, the library warns in the console and uses its default map; check the warning rather than assuming that an empty scene is your intended level.

Expected results: the player collects a beeper in level one. After level two loads, that same player remains at [3, 2], facing east, holding 1. A wall in the new map blocks its front. Loading replaces walls, beeper stacks, and custom items, but preserves existing robots and listeners. It does not reset player position, facing, inventory, or your score variables.

For a map already packaged in the JAR, `Arena.loadMap(Arena.DEFAULT_MAP)` selects the built-in default by its resource name. Local maps use file paths; this example keeps them beside the Java file for a predictable student setup.

The outer XML tag is still `world` for file compatibility. A wall's length expands into that many unit segments. Horizontal segments advance along x; vertical segments advance along y. You do not need a second XML library to load these files.



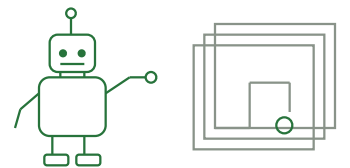
Lesson 6: Experiments

1. Change level one's wall length. Predict the affected cells and check sensors.
2. Change a beeper count. Verify what is collected before the level transition.
3. Add a robot tag and explain why the new robot appears alongside the player.
4. Design a level-transition method that chooses explicitly whether to keep the player or remove it with `explode()` and construct a replacement.

Show what you learned

List the state that survives a load and the state that is replaced. Explain where your two files live and how the program finds them. Show that a new wall can affect an existing robot; avoid spawning it inside a wall layout you have not checked.

Teacher prompt: distinguish the map name requested by `mapLoaded` from proof that a missing-file fallback did not occur. Do not teach an automatic reset rule that the current implementation does not provide.



Lesson 7: Draw a goal and a different player

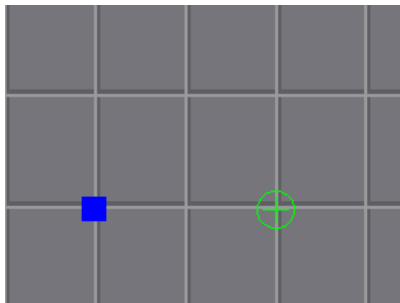
New capability: Custom Items and Rendering

Your game might need a target, a treasure marker, or a different-looking player. Give an object responsibility for drawing itself while keeping game decisions in the main loop.

Self-rendering items

An Item subclass supplies its drawing through render. The arena provides pixel coordinates; the item retains a grid location. Drawing describes appearance. The main loop supplies game behavior.

Before reaching the goal



render draws
the shapes

game loop
removes goal



After reaching the goal



Details from the actual example classes: the blue square is the player; the green target disappears when reached.

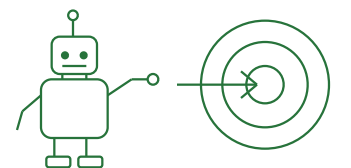
Predict and run

Compile `CustomItems.java`. A goal at `[4, 2]` is drawn as an outlined green target. A Robot subclass draws itself as a small blue square. After two eastward moves from `[2, 2]`, the player reaches the goal; the main loop removes it and prints Reached the goal at `[4, 2]`. Use slow or STEP pacing to watch the target before it disappears.

```
private static class Goal extends Item {
    Goal(int x, int y) { super(x, y); }
    public void render(Graphics g, int x, int y) {
        g.drawOval(x - 12, y - 12, 24, 24);
    }
}
```

Item is abstract. Its render method is the drawing contract, and your subclass must supply an implementation. Register a completed goal with `Arena.getModel().addUserItem(goal)` and remove it with `removeUserItem(goal)`. A custom item is decorative until you write its behavior: a Goal does not become a wall, beeper, or collision automatically.

The `x` and `y` passed to render are pixel coordinates chosen by the arena. The Item's `getX()` and `getY()` are grid coordinates. Drawing may happen many times on Swing's painting thread: do not move robots, change scores, pause, or modify the arena inside render.



Compare grid positions with `player.getLocation().equals(goal.getLocation())`. Using `==` would ask whether two references identify the same `Location` object. Read locations for decisions; use a subclass's protected `updatePosition` method when designing an item that can relocate, rather than changing another object's returned `Location`.

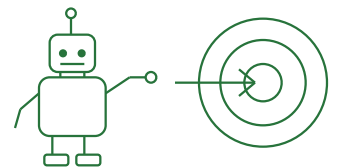
Exercises

1. Change the outline or marker size. Show that appearance does not change location.
2. Keep the goal after the player arrives and make the main loop announce it once.
3. Override the player's drawing with a circle, triangle, or shield. Keep robot movement and sensors working. A custom render override must decide how to show direction; it does not receive the default robot icon automatically.
4. Track additions and removals in a listener. The existing `userItemDropped` callback name means removal of a custom item.

Show what you learned

Explain abstract `Item` versus concrete `Goal`, pixel versus grid coordinates, and rendering versus a game rule. Demonstrate one custom item and one custom robot appearance without introducing side effects in rendering.

Teacher extension: ask what a treasure interface would promise to the game. Start with a small behavior such as an integer point value; do not make every visual item responsible for the entire game loop.



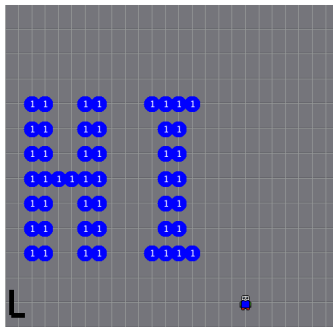
Lesson 8: A robot that draws letters

New capability: Robot Lettering

Use an existing Robot subclass to turn a short message into beeper art. Then inspect how a larger behavior is composed from familiar robot actions.

Lettering and infinite supplies

AlphaBot composes ordinary moves and drops into bitmap letters. Its unlimited inventory uses `BeeperStack.INFINITY`, a special value. The current lettering routine supports ASCII characters on a clear route.



One instruction, many ordinary actions

`sign.say("HI")`
AlphaBot composes movement and beeper drops.
`BeeperStack.INFINITY` supplies unlimited beepers.
The lettering route needs room and clear cells.

AlphaBot writes HI with blue beepers. The named constant identifies an unlimited supply.

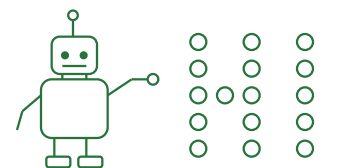
Predict and run

Compile `RobotLettering.java`. It imports AlphaBot from `org.loudouncodes.jkarel.demo`, opens a blank 24-by-12 arena, sets a fast pace, and writes HI in blue beepers.

```
AlphaBot sign = new AlphaBot();
sign.setColor(Color.BLUE);
sign.say("HI");
```

AlphaBot starts at `[2, 1]`, facing north, with `BeeperStack.INFINITY` beepers. Its inventory stays infinite after drawing. This sentinel represents an unlimited supply; it is not an ordinary negative inventory. A robot can use an infinite starting inventory, and a map can contain an infinite stack.

Each character is drawn using an 8-by-8 bitmap. Start with uppercase ASCII letters and a short message on an empty arena. The current font table accepts characters with codes 0 through 127; other characters are not supported. Walls can obstruct the drawing route, and the library does not expand the visible grid for a longer message automatically. Increase its width before drawing more letters.



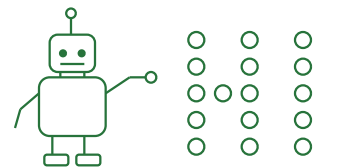
Lesson 8: Investigate and extend

1. Write a two-letter initial or short ASCII word. Predict the space it needs.
2. Change color between calls to `say`. Existing letters keep their stack colors.
3. Read `say`, `drawLetter`, and `scanLine` in `AlphaBot`. Explain how one public operation delegates to several small private methods and ordinary robot verbs.
4. Sketch an 8-by-8 symbol on paper, then write your own small drawing robot for a trophy or team emblem. Begin with loops and beepers before bit operations.

Show what you learned

Show your message, explain why the inventory does not run out, and trace one part of the drawing into moves and drops. Identify a limitation of `AlphaBot` and a change you would need before using it as a general message renderer.

Teacher prompt: distinguish an unlimited inventory from a scoreboard's numeric count. This is a worked example of inheritance and decomposition, not a requirement that students use the font-table implementation in their own game.



Lesson 9: An interface that describes a map

New concept: MapDataSource Interfaces (Design Exercise)

Advanced design extension. You know how to build an arena and observe its events. Now separate a description of a map from the code that installs it.

Data-source interfaces

An interface specifies what information an object supplies. Different implementations can describe maps. Here a student-written adapter installs that description; the library does not yet load MapDataSource objects directly.



Two implementations, one contract, one adapter. The library does not yet load this interface directly.

Read the contract

MapDataSource declares four methods: width, height, beeper stacks, and walls. An implementation could describe a fixed room or a generated maze. A caller can use the interface without needing to know how that description was produced.

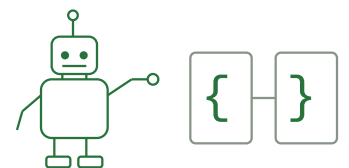
This is a design seam in the current library. `Arena.loadMap` accepts a `String`; there is no `Arena.loadMap(MapDataSource)` overload, and the XML loader is not wired through this interface. This exercise supplies its own small adapter, using operations you have already practiced. It does not change the library.

Predict and run

Compile `DescribeAMap.java`. `TrainingMap` implements the interface, while `install` accepts any `MapDataSource`.

```
private static void install(MapDataSource source) {
    Arena.openDefaultMap();
    Arena.setSize(source.getWidth(), source.getHeight());
    for (Wall wall : source.getWalls())
        Arena.getModel().addWall(wall);
    for (BeeperStack stack : source.getBeeperStacks().values())
        Arena.getModel().putBeeper(stack.getLocation(),
            stack.getBeeper(), stack.getColor());
}
```

Expected result: an 8-by-6 map with one north wall at [3, 3] and two beepers at [4, 2]. The player walks there and collects both, ending with inventory 2. Setup finishes before the player is constructed. The adapter's map



reset has the same robot-preservation semantics as the map-loading lesson.

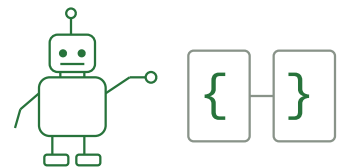
Make a second implementation

1. Write `SupplyRoom` implements `MapDataSource` with a different size and layout. Install it using the same method; do not add a second installation method.
2. Explain the `Map<Location, BeeperStack>` and `List<Wall>` return types. The `Map` is a Java collection indexed by locations; it is not the whole game map.
3. Choose a policy for returned collections: newly built descriptions, snapshots, or read-only views. Explain whether callers can change your source accidentally.
4. Sketch a future library overload and its tests: colored stacks, long walls, existing robots, and listener notifications. Discuss before implementing it.

Show what you learned

Demonstrate two map-source implementations using one adapter. Explain an interface promise, an implementation choice, and an adapter responsibility. Identify exactly which part is supported by the current library and which part you wrote.

Teacher prompt: compare this interface, which is queried for data, with `ArenaListener`, whose methods receive notifications. Keep this example small; leave maze generation and refactoring for later work.



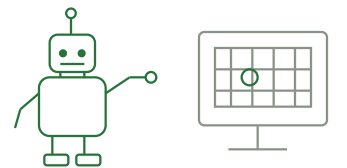
Complete example programs

Copy each listing into a file with the name shown and run it separately in jGRASP. For MapStages, also copy both map files from the listings below into that working folder. Student starter folders contain focused TODOs; these appendix listings are complete references. The same sources are in examples/java and examples/maps.

WelcomeArena.java

```
// GPLv3; see LICENSE.TXT and NON-ENDORSEMENT.TXT for Bock contributions.
import org.loudouncodes.jkarel.*;

public class WelcomeArena {
    public static void main(String[] args) {
        // Arena calls prepare the shared world before we add a robot.
        Arena.openDefaultMap();
        Arena.setSize(8, 6);
        Arena.setPace(Pacing.FAST);
        // Constructor arguments: column, row, facing, starting inventory.
        Robot explorer = new Robot(2, 2, Direction.EAST, 2);
        // Robot calls change this robot; the dropped beeper stays behind.
        explorer.putBeeper();
        explorer.move();
        explorer.turnLeft();
        explorer.move();
        System.out.println("Location: " + explorer.getLocation());
        System.out.println("Direction: " + explorer.getDirection());
        System.out.println("Inventory: " + explorer.getBeepers());
    }
}
```



TeamTrails.java

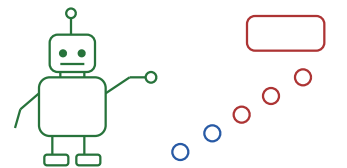
```
// GPLv3; see LICENSE.TXT and NON-ENDORSEMENT.TXT for Bock contributions.
import java.awt.Color;
import org.loudouncodes.jkarel.*;

/** Extension lab 1: two teams leave colored trails inside a student-built arena. */
public class TeamTrails {
    public static void main(String[] args) {
        Arena.openDefaultMap();
        Arena.setSize(8, 6);
        Arena.setPace(Pacing.FAST);
        buildWalls();

        Robot red = new Robot(2, 2, Direction.EAST, 5);
        Robot blue = new Robot(2, 4, Direction.EAST, 5);
        // Drops use the robot color; team identity is stored in these variables.
        red.setColor(Color.RED);
        blue.setColor(Color.BLUE);

        // Five drops need only four moves: both trails end in column 6.
        for (int step = 0; step < 5; step++) {
            red.putBeeper();
            blue.putBeeper();
            if (step < 4) {
                if (red.frontIsClear()) red.move();
                if (blue.frontIsClear()) blue.move();
            }
        }
        System.out.println("Both teams left five beepers, from column 2 through column 6.");
    }

    private static void buildWalls() {
        // setSize changes the visible grid. These loops create the boundaries.
        for (int x = 1; x <= 8; x++) {
            Arena.addSouthWall(x, 1);
            Arena.addNorthWall(x, 6);
        }
        for (int y = 1; y <= 6; y++) {
            Arena.addWestWall(1, y);
            Arena.addEastWall(8, y);
        }
        // A divider between the two teams' lanes.
        for (int x = 2; x <= 6; x++) Arena.addNorthWall(x, 2);
    }
}
```



EventScoreboard.java

```
// GPLv3; see LICENSE.TXT and NON-ENDORSEMENT.TXT for Bock contributions.
import java.awt.Color;
import org.loudouncodes.jkarel.*;

/** Extension lab 2: observe moves and pickups without putting scoring in Robot. */
public class EventScoreboard {
    public static void main(String[] args) {
        Arena.openDefaultMap();
        Arena.setSize(8, 6);
        Arena.setPace(Pacing.FAST);
        for (int x = 2; x <= 6; x++) Arena.addNorthWall(x, 2);
        Arena.addEastWall(6, 2);
        Arena.addEastWall(6, 4);
        Arena.addBeeper(3, 2); Arena.addBeeper(4, 2);
        Arena.addBeeper(3, 4); Arena.addBeeper(4, 4);

        Robot red = new Robot(2, 2, Direction.EAST, 0);
        Robot blue = new Robot(2, 4, Direction.EAST, 0);
        red.setColor(Color.RED); blue.setColor(Color.BLUE);
        Scoreboard scores = new Scoreboard(red, blue);
        // Register before actions so the scoreboard receives their callbacks.
        Arena.addListener(scores);
        try {
            // The main program controls movement; the listener only observes it.
            for (int turn = 0; turn < 4; turn++) {
                takeTurn(red);
                takeTurn(blue);
            }
            scores.printResult();
        } finally {
            // Remove the observer even if an action throws an exception.
            Arena.removeListener(scores);
        }
    }

    private static void takeTurn(Robot robot) {
        if (robot.frontIsClear()) robot.move();
        if (robot.nextToABeeper()) robot.pickBeeper();
    }
}
```



EventScoreboard.java: listener implementation (continued)

```
private static class Scoreboard implements ArenaListener {
    private final Robot red;
    private final Robot blue;
    private int redScore;
    private int blueScore;
    private int moves;

    Scoreboard(Robot red, Robot blue) {
        this.red = red;
        this.blue = blue;
    }

    @Override
    public void robotMoved(Robot robot) {
        if (robot == red || robot == blue) moves++;
    }

    // Object identity identifies the player, even if its color changes.
    // The pickup has already changed the inventory when this is called.
    @Override
    public void beeperPickedUp(Robot robot) {
        if (robot == red) redScore++;
        if (robot == blue) blueScore++;
        System.out.println("Red: " + redScore + "   Blue: " + blueScore);
    }

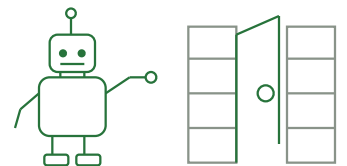
    void printResult() {
        System.out.println("Final: Red " + redScore + ", Blue " + blueScore
            + ", moves " + moves);
    }
}
```



RoomBuilder.java

// GPLv3; see LICENSE.TXT and NON-ENDORSEMENT.TXT for Bock contributions.
import org.loudouncodes.jkarel.*;

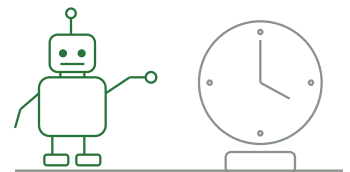
```
public class RoomBuilder {  
    public static void main(String[] args) {  
        Arena.openDefaultMap();  
        Arena.setSize(8, 6);  
        Arena.setPace(Pacing.FAST);  
        // A visible grid size does not build walls; these loops do.  
        for (int x = 1; x <= 8; x++) {  
            Arena.addSouthWall(x, 1);  
            Arena.addNorthWall(x, 6);  
        }  
        for (int y = 1; y <= 6; y++) {  
            Arena.addWestWall(1, y);  
            Arena.addEastWall(8, y);  
        }  
        // Leave a gap in the divider, then fill it with a removable door.  
        for (int y = 2; y <= 5; y++) {  
            if (y != 3) Arena.addEastWall(4, y);  
        }  
        // Keep this exact object so we can remove this wall later.  
        Wall door = new Wall(4, 3, Arena.VERTICAL);  
        Arena.getModel().addWall(door);  
        Arena.addBeepers(5, 3, 3);  
        Robot player = new Robot(4, 3, Direction.EAST, 0);  
        System.out.println("Door closed, front clear: " + player.frontIsClear());  
        // Opening the door changes the sensor result before the next move.  
        Arena.getModel().removeWall(door);  
        if (player.frontIsClear()) player.move();  
        if (player.nextToABeeper()) player.pickBeeper();  
        System.out.println("Player " + player.getLocation() + ", inventory " + player.getBeepers());  
    }  
}
```



PaceProbe.java

```
// GPLv3; see LICENSE.TXT and NON-ENDORSEMENT.TXT for Bock contributions.
import org.loudouncodes.jkarel.*;

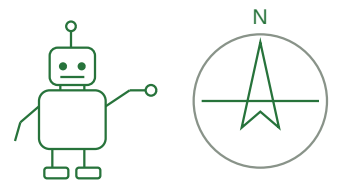
public class PaceProbe {
    public static void main(String[] args) {
        Arena.openDefaultMap();
        Arena.setPace(Pacing.FAST);
        Robot robot = new Robot(3, 3, Direction.EAST, 0);
        // Use --auto for an unattended graphical demonstration.
        boolean auto = args.length > 0 && args[0].equals("--auto");
        // STEP pauses after an action; Enter lets execution continue.
        Arena.setPace(auto ? Pacing.FAST : Pacing.STEP);
        System.out.println("After each action, press Enter in the run console.");
        robot.move();
        System.out.println(robot.getLocation() + " " + robot.getDirection());
        // Turning changes facing while preserving the location.
        robot.turnLeft();
        System.out.println(robot.getLocation() + " " + robot.getDirection());
        robot.move();
        System.out.println(robot.getLocation() + " " + robot.getDirection());
        Arena.setPace(Pacing.FAST);
    }
}
```



ScoutMoves.java

```
// GPLv3; see LICENSE.TXT and NON-ENDORSEMENT.TXT for Bock contributions.  
import org.loudouncodes.jkarel.*;
```

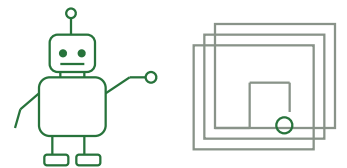
```
public class ScoutMoves {  
    private static class Scout extends Robot {  
        Scout() { super(3, 3, Direction.NORTH, 0); }  
        public void turnRight() { turnLeft(); turnLeft(); turnLeft(); }  
        // A subclass can call protected backUp; check the rear first.  
        public void retreat() { if (backIsClear()) backUp(); }  
    }  
    public static void main(String[] args) {  
        Arena.openDefaultMap();  
        Arena.setPace(Pacing.FAST);  
        Arena.addWestWall(3, 3);  
        Scout scout = new Scout();  
        System.out.println("Left clear: " + scout.leftIsClear());  
        // This computes a direction value without turning the scout.  
        Direction planned = scout.getDirection().right();  
        System.out.println("Planned direction: " + planned);  
        scout.turnRight();  
        if (scout.frontIsClear()) scout.move();  
        // Move backward without changing the eastward facing.  
        scout.retreat();  
        System.out.println("Scout " + scout.getLocation() + " " + scout.getDirection());  
    }  
}
```



MapStages.java

```
// GPLv3; see LICENSE.TXT and NON-ENDORSEMENT.TXT for Bock contributions.  
import org.loudouncodes.jkarel.*;
```

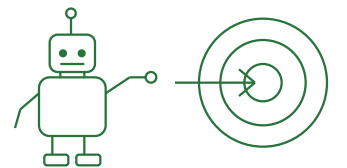
```
public class MapStages {  
    public static void main(String[] args) {  
        // In jGRASP, keep both map files beside this program.  
        String mapFolder = args.length == 0 ? "." : args[0];  
        Arena.openDefaultMap();  
        Arena.setPace(Pacing.FAST);  
        Robot player = new Robot(2, 2, Direction.EAST, 0);  
        // An anonymous listener needs to override only the callback we use.  
        ArenaListener observer = new ArenaListener() {  
            public void mapLoaded(String name) { System.out.println("Loaded: " + name); }  
        };  
        Arena.addListener(observer);  
        try {  
            Arena.loadMap(mapFolder + "/stage-one.map");  
            if (player.frontIsClear()) player.move();  
            if (player.nextToABeeper()) player.pickBeeper();  
            // Reloading replaces map contents but preserves this player.  
            // Its position, facing, and inventory survive the transition.  
            Arena.loadMap(mapFolder + "/stage-two.map");  
            System.out.println("Same player: " + player.getLocation() + ", inventory " + player.getBeepers());  
            System.out.println("New wall blocks front: " + !player.frontIsClear());  
        } finally {  
            Arena.removeListener(observer);  
        }  
    }  
}
```



CustomItems.java

```
// GPLv3; see LICENSE.TXT and NON-ENDORSEMENT.TXT for Bock contributions.
import java.awt.Color;
import java.awt.Graphics;
import org.loudouncodes.jkarel.*;

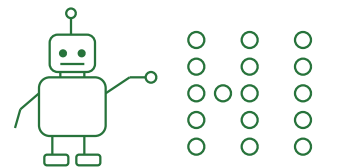
public class CustomItems {
    private static class Goal extends Item {
        Goal(int x, int y) { super(x, y); }
        // x and y are pixel centers supplied by the arena, not grid cells.
        public void render(Graphics graphics, int x, int y) {
            graphics.setColor(Color.GREEN);
            graphics.drawOval(x - 12, y - 12, 24, 24);
            graphics.drawLine(x - 8, y, x + 8, y);
            graphics.drawLine(x, y - 8, x, y + 8);
        }
    }
    // Override drawing while inheriting the normal robot actions and sensors.
    private static class Player extends Robot {
        Player() { super(2, 2, Direction.EAST, 0); }
        // x and y are pixel centers supplied by the arena, not grid cells.
        public void render(Graphics graphics, int x, int y) {
            graphics.setColor(getColor());
            graphics.fillRect(x - 8, y - 8, 16, 16);
        }
    }
    public static void main(String[] args) {
        Arena.openDefaultMap();
        Arena.setPace(Pacing.FAST);
        Goal goal = new Goal(4, 2);
        Arena.getModel().addUserItem(goal);
        Player player = new Player();
        player.setColor(Color.BLUE);
        for (int turn = 0; turn < 2; turn++) {
            if (player.frontIsClear()) player.move();
            // The game loop decides when the goal is reached; drawing does not.
            if (player.getLocation().equals(goal.getLocation())) {
                Arena.getModel().removeUserItem(goal);
                System.out.println("Reached the goal at " + player.getLocation());
            }
        }
    }
}
```



RobotLettering.java

```
// GPLv3; see LICENSE.TXT and NON-ENDORSEMENT.TXT for Bock contributions.
import java.awt.Color;
import org.loudouncodes.jkarel.*;
import org.loudouncodes.jkarel.demo.AlphaBot;

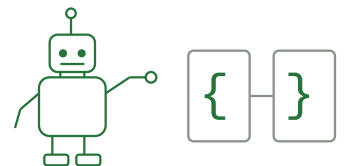
public class RobotLettering {
    public static void main(String[] args) {
        Arena.openDefaultMap();
        // Leave a clear route and enough room for the lettering routine.
        Arena.setSize(24, 12);
        Arena.setPace(Pacing.LUDICRUS);
        AlphaBot sign = new AlphaBot();
        sign.setColor(Color.BLUE);
        // AlphaBot draws ASCII letters with moves and colored beeper drops.
        sign.say("HI");
        // Unlimited supplies use a named sentinel, not an ordinary count.
        System.out.println("Infinite inventory: " + (sign.getBeeperCount() == BeeperStack.INFINITY));
    }
}
```



DescribeAMap.java

```
// GPLv3; see LICENSE.TXT and NON-ENDORSEMENT.TXT for Bock contributions.
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import org.loudouncodes.jkarel.*;

public class DescribeAMap {
    // The interface describes map data independently of installation.
    private static class TrainingMap implements MapDataSource {
        public int getWidth() { return 8; }
        public int getHeight() { return 6; }
        public Map<Location, BeeperStack> getBeepers() {
            // Return a fresh collection indexed by grid locations.
            Map<Location, BeeperStack> beepers = new HashMap<>();
            beepers.put(new Location(4, 2), new BeeperStack(4, 2, 2));
            return beepers;
        }
        public List<Wall> getWalls() {
            List<Wall> walls = new ArrayList<>();
            walls.add(new Wall(3, 3, Arena.HORIZONTAL));
            return walls;
        }
    }
    // This is OUR adapter. Arena has no loadMap(MapDataSource) overload yet.
    private static void install(MapDataSource source) {
        Arena.openDefaultMap();
        Arena.setSize(source.getWidth(), source.getHeight());
        // Translate the interface data into existing model operations.
        for (Wall wall : source.getWalls()) Arena.getModel().addWall(wall);
        for (BeeperStack stack : source.getBeepers().values())
            Arena.getModel().putBeepers(stack.getLocation(), stack.getBeepers(), stack.getColor());
    }
    public static void main(String[] args) {
        Arena.setPace(Pacing.FAST);
        // Finish constructing the scene before creating the player.
        install(new TrainingMap());
        Robot player = new Robot(2, 2, Direction.EAST, 0);
        player.move(); player.move();
        while (player.nextToABeeper()) player.pickBeeper();
        System.out.println("Player " + player.getLocation() + ", inventory " + player.getBeepers());
    }
}
```



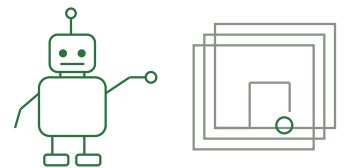
Map files for the level-changing lesson

stage-one.map

```
<world>
  <properties><defaultSize width='8' height='6'></properties>
  <objects>
    <beeper x='3' y='2' num='1'>
    <wall x='2' y='3' length='3' style='horizontal'>
  </objects>
</world>
```

stage-two.map

```
<world>
  <properties><defaultSize width='8' height='6'></properties>
  <objects>
    <wall x='3' y='2' length='2' style='vertical'>
    <beeper x='5' y='4' num='3'>
  </objects>
</world>
```



Teacher notes and sources

Teacher acceptance

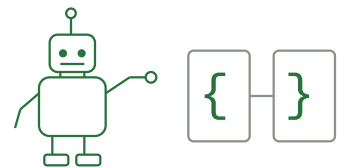
Before giving the JAR to students, run the examples in the actual classroom jGRASP installation. Check red and blue robots/trails in the first lab; check pickup scores and the final eight-move count in the second. Check STEP interactively in the run console and check the map files from the student working folder. Java installation and visual display are classroom checks, separate from the automated Ant tests.

References and license

Lesson text and original illustrations: © 2026 Bock / LoudounCodes, [CC BY 4.0](#). You may share and adapt them with credit, a license link, and an indication of changes. Credit does not imply endorsement.

The library and Java examples remain [GPLv3](#). Arena screenshots follow the library's license; the LoudounCodes logo is excluded from the CC BY grant. See the licensing guide (LICENSING.md in the classroom ZIP) for details.

For jGRASP setup, see the official [installation tutorial](#), [chapter 1: PATH/CLASSPATH](#) and [project tutorial](#), [chapter 7](#). These references describe version 1.8.7 (2009); menus may vary with the version installed in your classroom.



This page intentionally left blank.

Feature coverage and teaching boundaries

Use this guide to choose a lesson for a project or connect a feature to the Java concepts it introduces.

Feature or change	Lesson	Actual teaching surface
Display → Arena; Coordinate → Location; simpler package	0	Arena facade, Robot location accessors, <code>org.loudouncodes.jkarel imports</code>
Typed direction rather than numeric constants	0, 5	Direction enum; left/right/behind compute values
Robot and whole-stack beeper colors	1	<code>setColor/getColor</code> ; colored drops; existing stack color survives pickup
ArenaListener behavior interface and default methods	2	<code>addListener/removeListener</code> ; synchronous notifications; score outside Robot
Programmatic walls and beepers, removable doors	3	directional wall calls; <code>addBeeper/addBeepers</code> ; stored Wall reference
Named pacing and STEP	4	Pacing enum; Enter in run console; setup and action pause semantics
Protected backup and reusable subclasses	5	<code>backUp</code> inside Scout; <code>backIsClear</code> ; position versus facing
Runtime map reload; bundled and local maps	6	<code>loadMap(String)</code> ; existing player/listener persistence; XML wall length
Self-rendering objects and generic user items	7	abstract Item; <code>add/removeUserItem</code> ; Robot.render override; grid versus pixels
Letter-drawing robot and infinite supply ownership	8	<code>demo.AlphaBot.say</code> ; <code>BeeperStack.INFINITY</code> ; ASCII and layout limits
A data-source interface for future maps	9	MapDataSource descriptor; explicit student-written installation adapter



Teaching notes

Begin with Lesson 0, then choose the features a student's project needs. Familiar robot actions, sensors, subclassing, and game loops remain part of the working vocabulary.

The event lesson introduces movement and pickup scoring, then explores drops and unregistering. Later lessons use map-load events and custom-item removal. Callbacks run synchronously; students should keep game loops outside listeners. The library does not provide timer or keyboard-input events.

Lesson 9 is an advanced design exercise. The MapDataSource interface describes a map, but the XML loader does not consume it directly. Students write an adapter in their example program and use it with two implementations.

